



GeoVISTA *Studio*: a codeless visual programming environment for geoscientific data analysis and visualization[☆]

Masahiro Takatsuka*, Mark Gahegan

GeoVISTA Center, Department of Geography, The Pennsylvania State University, University Park, PA 16802-5011, USA

Received 14 February 2001; received in revised form 9 September 2001; accepted 1 January 2002

Abstract

The fundamental goal of the GeoVISTA *Studio* project is to improve geoscientific analysis by providing an environment that operationally integrates a wide range of analysis activities, including those both computationally and visually based. Improving the infrastructure used in analysis has far-reaching potential to better integrate human-based and computationally based expertise, and so ultimately improve scientific outcomes. To address these challenges, some difficult system design and software engineering problems must be overcome.

This paper illustrates the design of a component-oriented system, GeoVISTA *Studio*, as a means to overcome such difficulties by using state-of-the-art component-based software engineering techniques. Advantages described include: ease of program construction (visual programming), an open (non-proprietary) architecture, simple component-based integration and advanced deployment methods. This versatility has the potential to change the nature of systems development for the geosciences, providing better mechanisms to coordinate complex functionality, and as a consequence, to improve analysis by closer integration of software tools and better engagement of the human expert. Two example applications are presented to illustrate the potential of the *Studio* environment for exploring and better understanding large, complex geographical datasets and for supporting complex visual and computational analysis.

© 2002 Elsevier Science Ltd. All rights reserved.

Keywords: Visual programming; Exploratory data analysis (EDA); Knowledge construction; Java; Component-oriented programming (COP)

1. Introduction

Despite extensive efforts in quantification, many branches of science, and the Earth sciences in particular, remain *non-axiomatic*; it is not possible to deduce all outcomes from known laws. Science must, therefore, be approached in a manner that encourages the creation or discovery of new knowledge (Baker, 1999). Accordingly, scientists need an analytical environment that encourages the positing, development, testing and evaluation

of new hypotheses concerning the structure of complex Earth systems (Valdez-Perez, 1999). GeoVISTA *Studio* (referred to as *Studio* in the rest of the paper) is such an environment; designed to support visual and computational exploration and analysis of complex geoscientific datasets as collaboration between computationally based and human-based expertise.

The remainder of this section describes approaches to geoscientific analysis and points out some current problems and challenges, then Section 2 provides an overview of *Studio*, with practical use of the system and deployment options described in Section 3. Section 4 details two example applications constructed using *Studio*. Conclusions are then presented in Section 5.

[☆]Code available from: <http://www.geovistastudio.psu.edu/>

*Corresponding author.

E-mail addresses: masa@psu.edu (M. Takatsuka), mark@geog.psu.edu (M. Gahegan).

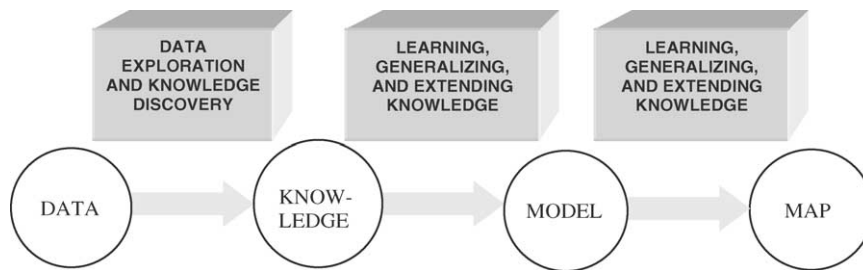


Fig. 1. Linearization of scientific activities result of functionality being compartmentalized within individual systems.

The following four sub-sections describe some of the motivation underlying the development of the *Studio* environment under the headings of: (1) integrating the various stages of analysis; (2) thinking visually; (3) dealing with increasing data complexity; and (4) incorporating scientific models.

1.1. Integrating the various stages of analysis

Scientific analysis employs a number of stages, including observation, data exploration, model construction, simulation, verification and communication of results (Hanson, 1958; Popper, 1959; Langley, 2000). These activities might begin with *abductive* tasks such as hypothesis formation and knowledge construction, through *inductive* tasks such as classification and learning from examples and ending with *deductive* systems that build deterministic models, which are common across the spectrum of physical sciences (Peirce, 1878). Baker (1999) provides an excellent account of these different forms of *inference*, from an Earth science perspective. However, there is no single package or system that currently supports these different types of inference in an integrated fashion; users must instead resort to a set of disparate (and often clumsy) programs that are difficult to connect together operationally (e.g. Breunig and Perkhoff, 1992; Abel et al., 1994; Gahegan, 1999) and that do not engage the knowledge of the expert in an efficient manner (MacEachren et al., 1999).

Such a state of affairs encourages an over-reliance on deductive reasoning and represents a considerable bottleneck in the advancement of our theoretical understanding of complex processes. For example, software packages such as Geographic Information Systems (GIS) provide limited functionality outside of the statistical deductive tradition. Consequently, they lack the flexibility and functionality required to support visualization and analysis targeted at exploring data and constructing knowledge (Haslett et al., 1991; Tang, 1992; Gahegan et al., 2000). They are instead aimed at

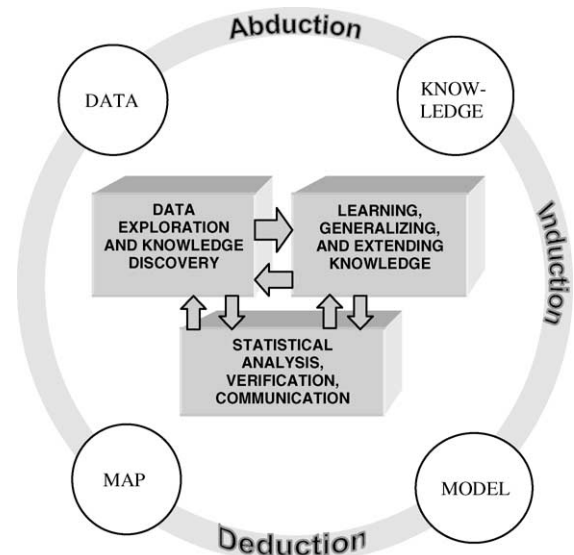


Fig. 2. Integrated environment helps to promote flow of analysis activities, one to another, in any direction.

the later stages of analysis and the cartographic presentation of final results.

This is a serious problem; creativity is stifled by separating observation from hypothesis generation, by separating data interpretation from data manipulation and by separating visual presentation of information from quantitative analysis. Current approaches to scientific analysis can often be characterized as largely linear and unidirectional—the *process* of data analysis must be established before data analysis is carried out, with feedback and revision being awkward to apply (see Fig. 1).

Alternatively, scientific activities would benefit from being addressed in parallel, then a formulated model can directly influence the form of analysis, evaluation can directly influence model refinement, and so forth. The resulting system might be conceptualized as a circle where the different inference mechanisms provide an integrated means of moving between inter-related activities, as shown in Fig. 2.

1.2. Thinking visually

Although computational analysis has become the mainstay of many sub-disciplines in the Earth sciences, the fact remains that the most comprehensive models and understanding are available exclusively from the human expert. The argument for providing visually based analysis tools to engage and apply this expertise is by now well established. For example, Card et al. (1999) describe a comprehensive range of successful information visualization applications and two recent special issues of Computers & Geosciences have been dedicated exclusively to exploratory spatial data analysis using visualization (e.g. MacEachren and Kraak, 1997). Risch et al. (1997) describe how the integrated use of multiple, concurrent visualization techniques can improve a user's understanding of complex and highly inter-related information, and go on to claim that "This approach enables powerful new forms of information analysis, while at the same time easing cognitive workloads by providing a visual context for the information under study."

One difficulty with current visualization systems is that they typically exclude more traditional forms of (quantitative) analysis, or at least make them difficult to amalgamate operationally. Here, we describe an environment that naturally integrates both visual and computationally based analysis tools, with the aim of achieving better synergy between the human domain expert and the machine.

1.3. Dealing with increasing data complexity

The data available to scientists continues to increase in complexity, in terms of measurement precision, number of observations and features observed. In fact, there is growing concern that current statistical approaches to analysis may not scale well to address aspects of this complexity (e.g. Elder and Pregibon, 1996; Glymour et al., 1997; Landgrebe, 2000; Gahegan, 2000), and therefore may need to be augmented with data mining, machine learning and visually based methods (Fayyad et al., 1996; Mitchell, 1997; Valdez-Perez, 1999). *Studio* provides various data analysis tools to support such activities as described in Section 4.

1.4. Interoperating scientific models

Despite substantial progress in geospatial interoperability, the exchange of scientific models among scientists remains frustratingly difficult. Standards for exchanging data are now relatively advanced (e.g. Kottman, 1998; Goodchild et al., 1999), but it is often forgotten that our data are only useful with appropriate analytical models, yet how to make these models interoperable is unknown (Goodchild, 2000). GeoVIS-

TA *Studio* provides an environment in which complex functionality can be linked together into models. Using JavaBeans technology, it is straightforward to extend these models or couple them to additional methods (Section 3.3 gives more detail). *Studio* also offers a simple means to 'wrap up' the assembled functionality into a working program (in the form of a JavaBeans component, an applet, or an application) that can be easily disseminated or deployed on the Internet (see Section 3.5). Such deployment mechanisms allow researchers to: formalise models (from a functional perspective), ensure repeatability, promote the sharing and exchange of models between collaborating scientists, and therefore to verify results independently. Each of these steps makes a valuable contribution to the improvement of the scientific process outlined previously.

2. Overview of GeoVISTA Studio

Many geoscientists would prefer to focus on actual data analysis and visualization and not on the underlying programming logic that provides these tools and facilitates their dynamic coordination. However, so far, the uses of machine learning, visualization and knowledge discovery make considerable demands on the computational expertise of the user. Furthermore, even if individual components can be mastered, there is little or no provision for their interaction. In order to allow the scientist to concentrate on the various stages of analysis outlined in Section 1.1, a system has to satisfy the following demands:

- offer many heterogeneous program components within a single environment;
- be easy to use but with the capability to construct complex functionality;
- allow rapid development and modification of applications, minimizing programming requirements;
- support the sharing and exchange of developed applications.

In addition, it would be advantageous to offer cross-platform support and Internet-based deployment.

These requirements have several implications. For a component (tool) to be independently developed and deployed, it has to be decoupled from the system and other components (Szyperski, 1999). If third parties independently develop components, a clear standard must dictate the component development, so that compatibility is assured. Such a standard is preferably *open*, i.e. generally available and accepted by the community of application developers (Johnson, 1994). For non-programmers to build these components into sophisticated applications, each component needs to be

easily manipulated and connected via a standardized interface. This implies the need for an easy-to-use visual programming environment, i.e. where components are assembled together using a visual interface instead of a programming language. Moreover, due to the recent success with distributed computing and the Internet, we can no longer assume that analysis and visualization applications will be run only on a single hardware platform; these applications must be executable in a heterogeneous computing environment—leading to the requirement for cross-platform support. Finally, a system must allow a program to develop, as the scientific process itself develops in response to deepening understanding (see Fig. 2).

Studio was designed and built to meet these demands. At its heart, *Studio* contains a component-oriented application construction system (called “*Builder*”). Developers utilize its visual programming environment to connect functionality together rapidly into a useful application. This visual programming environment allows codeless program development, and is a convenient way to construct rapid prototypes that might uncover or evaluate hypotheses, and couple together analysis tools in various configurations in the search for useful insight. Connections can be quickly made and quickly broken as the user explores the various ways of tackling a problem and analyzes their relative merits. This is important since there are as yet no rules as to how best to uncover useful geoscientific knowledge; the facilitation of knowledge discovery is itself an experiment.

2.1. Java and JavaBeans—underlying technology

Studio is built around the well-established, standard Java programming language (Joy et al., 2000) and JavaBeans component programming technology (Sun Microsystems Inc., 1997) forming a layered model as shown in Fig. 3. One of the main reasons for choosing Java was its support for Component-Oriented Programming (COP). It is known that C++ does not directly support the concept of COP (Szyperski, 1999). Moreover, Java has been accepted in various fields due to its many benefits, such as cross-platform support (the bottom layer in Fig. 3) and ease of use, as evidenced by the growing transition from C++ to Java.

Some may claim that Java programs execute more slowly than C/C++ programs, due to their execution within the Java Virtual Machine, rather than in the machine code of the host computer. However, performance penalties are not severe in the author’s experience (see Section 4). Furthermore, improvements to modern compiler technology, such as Just-In-Time (JIT) and Ahead-Of-Time (AOT) compilers (Sun Microsystems Inc., 2000c), offer good solutions to the speed problem.

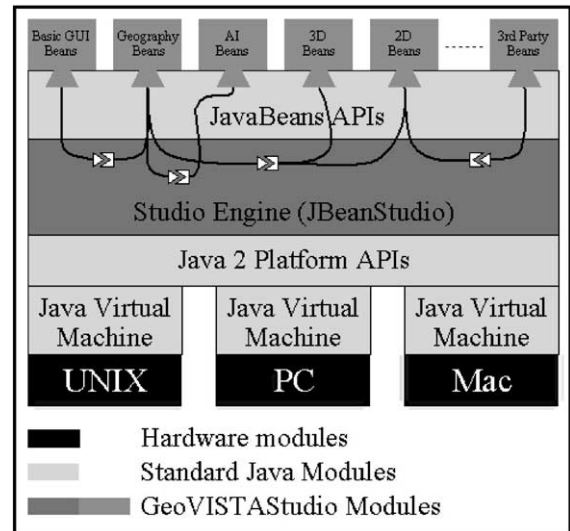


Fig. 3. Layered diagram of GeoVISTA *Studio* and other underlying technologies.

Java provides many advantages for this small penalty, the first among them being portable code.

A program in *Studio* is constructed from building blocks called components (see the connections made at the layer of the *Studio Engine* in Fig. 3). By combining many components, more complex programs can be developed. In software engineering, the software component is defined, as “...a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.” (Szyperski and Pfister, 1997). A component is usually required to be reusable; once it is written for a particular task or unified set of tasks, it should generalize to other similar tasks, saving time and effort in subsequent program development.

JavaBeans technology provides a standard Application Programming Interface (API) to make reusable component in the Java programming language. (*JavaBeans* refers to the API for beans and the related services provided by the JavaBeans component architecture, whereas *Java beans*, or simply *beans*, are the components made in accordance with the JavaBeans standard.) Like its underlying programming language, JavaBeans technology is also based on an open standard. As long as Java beans are created according to this standard, one can assume that they will work together with other beans created by different software vendors or individuals (see the top layer of Fig. 3). Beans will also run on many different computer platforms without the need for recompilation or any changes whatsoever to the under-

lying code. In summary, Java beans within *Studio* are (Weaver and Robertson, 1998, pp. 18–19):

- components that contain mechanisms for easy and consistent interaction with one another, regardless of where they were made;
- displayed by a builder (a visual programming system);
- customized interactively to modify their behavior and appearance;
- made interoperable with one another by use of events (described later);
- saved and reloaded in the same state as when they were last used. Hence, all of the settings and the data used in an application can be saved together, allowing exact replicas of experiments to be shared.

2.2. The user models

A program in *Studio* is constructed by connecting components to create a design. No programming is required; in fact *Studio* does not support code writing (the user can write the beans outside of *Studio*). This makes *Studio* different from other development tools in which a user has to write some code to produce applets or applications; hence, our users should find development less burdensome. Three distinct types of users are specifically catered for:

2.2.1. Tool users

This is the type of user whose primary concern is immediate and requires direct analysis and visualization of data, and also fast delivery (publishing) of simple interactive analysis and visualization results, for instance via the Internet. These users can quickly construct a data analysis and visualization program by connecting together existing tools.

2.2.2. Application developers

This type of user needs not only analysis and visualization results but also more sophisticated interactive and coordinated applications, perhaps to discover knowledge or formulate a scientific model (see Examples later in Section 4). They might build their own JavaBeans components where no current tools are

suitable for their tasks. These new tools would be quickly incorporated into *Studio* and tested.

2.2.3. Application users

This type of user would not use *Studio* directly. They would instead use the standalone applications or applets produced by *Studio*. Since *Studio* is based on Java open standard technology, applications produced can be deployed over the Internet (see Section 3.5) and so be available to a wide range of end-users. Applications of this type should prove especially useful for educational purposes, since application developers can define exactly what is shown to inexperienced users, and effectively remove all other details.

2.3. A typical work flow in GeoVISTA Studio

Regardless of the type of user, designing a program in *Studio* involves the following steps:

- (1) list all requirements to achieve the desired data analysis and visualization task;
- (2) choose the necessary components. (If desired components are not provided, those tools have to be made outside of *Studio*.);
- (3) place the selected components in the Design window or Graphical User Interface (GUI) window (see Fig. 4).
- (4) connect the components together according to the desired flow of data and control;
- (5) customize the components if necessary;
- (6) test the design in the GUI window;
- (7) save the design;
- (8) if a user plans to publish the design on the Internet or to distribute it as a standalone program, create an applet or an application using *Studio*'s deployment functionality (Section 3.5).

Since *Studio* utilizes a visual programming environment, it is able to unify the component-assembly and component-use environments. The GUI window is always “live” while a program is designed, so a design can always be changed as a consequence of using the constructed program and evaluating the results produced. Such seamless integration of design and user

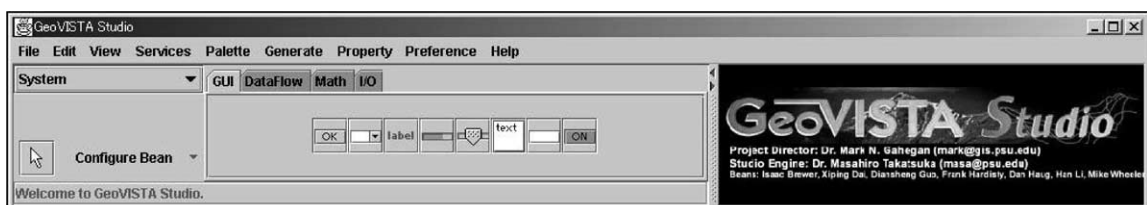


Fig. 4. GeoVISTA Studio main window showing basic bean folders and main system options.

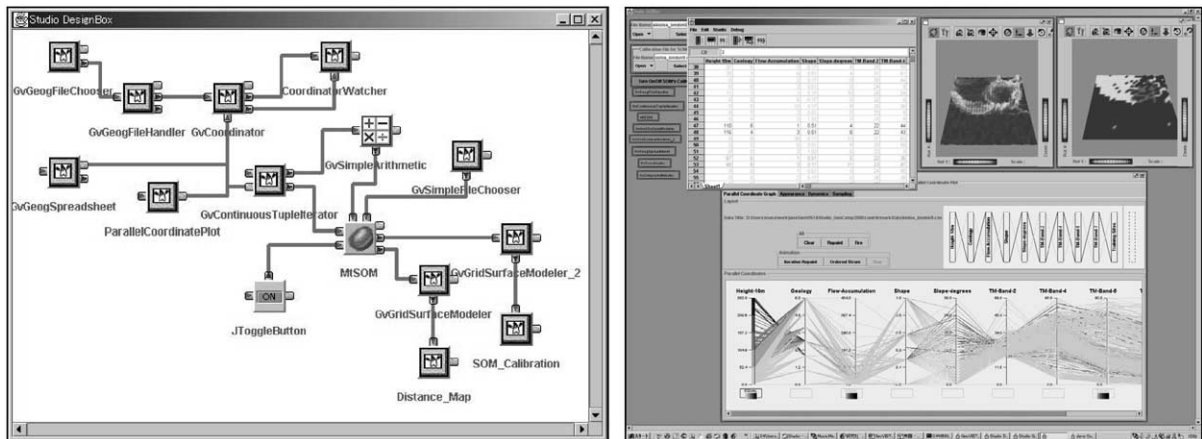


Fig. 5. Design window (left) contains a number of beans connected together to perform exploratory data analysis; GUI window (right) shows their respective visual interfaces.

environments enhances productivity since it allows rapid adaptation of the program in accordance with new requirements or new insights as knowledge is developed (Section 1.1).

2.4. Related works

Visual programming environments have been used successfully in many application areas. Especially in the field of 3-D visualization there are several good commercial and shareware systems such as AVS (Upson et al., 1990), IRIS Explorer, NAG (Numerical Algorithms Group, 2000) and IBM Data Explorer (IBM, 1995). In the field of general data analysis, LabVIEW (National Instruments Corp., 2001) is widely used via an excellent visual programming environment. Such products provide high-quality visualization or computation, but typically not both. Furthermore, these environments are generally ‘closed’, so developed components have to conform to proprietary structures and standards, locking the developer in and forming a barrier to interoperability.

More recently, a handful of Java-based visual programming systems have been developed, specifically to provide a more open environment. Two examples are the *BeanBox* (Sun Microsystems Inc., 2000d) in the JavaBeans Developer Kit (BDK) and *Java Studio™* from Sun Microsystems (although development and sale was discontinued in 1999). The BDK was developed as a reference implementation of JavaBeans API and a JavaBeans based *builder*. It is able to test basic functionalities of Java beans once they are developed, but does not provide enough visually based programming support to be more widely useful. Java Studio, on the other hand, provided an excellent visual and codeless, COP environment. It included a special layer

to provide more high-level codeless programming functions. However, this special layer—a kind of framework for visual programming—made Java Studio yet another proprietary environment. Nevertheless, it inspired the studio developers greatly in terms of the foresight shown in its design.

3. GeoVISTA Studio basics

This section contains an operational view of *Studio*, so that readers may gain a sense of what it is like to use. Of course, the best way to gain such insight is to download the *Studio* (<http://www.geovistastudio.psu.edu/>) and try it out for yourself! The following tools and functions are described:

- JavaBeans components and palettes;
- procedures for modifying the components to suit the needs of a particular design;
- windows for designing a program;
- online help system, which automatically integrates the help files of installed JavaBeans components.

3.1. Studio window

When *Studio* is first executed, it opens three windows: the Main window, the Design window and the GUI window. The Main window contains various menus and JavaBeans component palettes (see Fig. 4). The Design window is where a user constructs the design by placing components and “wiring” them together. The GUI window contains the graphical interfaces of all beans in current use. This window is always “live”; therefore, the user can always check how the design works thus far while constructing a program (see Fig. 5).

3.2. Palette and JavaBean components

By default, *Studio* provides several components that can be used “right out of the box”. Components in *Studio* are organized in folders within the component palette on the Main window (shown in Fig. 4). *Studio* is capable of loading many different palettes, each of which contains many folders (see Section A.2). Both palettes and folders are customizable; a user can modify them or even create new palettes or folders from the “Palette” menu.

The components in the palette are small pre-coded working programs. Hence, for instance, a 3-D viewer of a grid surface can be created without writing a single line of code. A data reader, grid surface modeler and 3-D renderer exist already in the 3-D palette, and they can be assembled together using only the mouse. However, in order to make the selected components suit a specific need, the user might want to change or modify properties such as appearance and functionality by customizing; the following section explains how to achieve this.

3.2.1. Property Editor

By using the Property Editor, a user can customize the available functionalities and appearance of a component in the Design and GUI window.

There are five different types of customization available in the Property Editor: Customize, Input, Output, Callback and Property.

1. *Customize*—If the creator of a bean component provided a special customizing program for it, that program will be displayed in this section. (The JavaBeans API provides a standard interface to create a customized program for a bean.)
2. *Input*—This section allows a user to specify which input methods should be exposed (made visible to the users of the bean) in the Design window as input connectors.
3. *Output*—Likewise, this section allows a user to specify which event methods should be exposed in the Design window as output connectors.
4. *Callback*—When a message (event) arrives at the input connector from the source component of the message, the destination component can initiate a method in the source component in order to obtain the information necessary to carry out its task. The source component’s method, which is being called from the destination component, is termed a callback method. This section allows a user to specify which methods should be exposed as callback methods in the Adapter Wizard (described later in Section 3.3).
5. *Property*—This section allows a user to customize the accessible properties of a component, such as back-

ground color, default font, and default size, using the standard JavaBeans API.

Each instance of a component can be customized differently, even if all instances are created from the same component. *Studio* also provides the Property Editor to customize the default settings of a component. Selecting the component in the palette and pressing the “Configure Bean” button invokes this editor, allowing the user to set the description and tool-tip comments for the component, as well as the default input and output methods, and the default callback methods.

3.2.2. Importing components

Any components can be used in *Studio*, provided they are created in accordance with Java and JavaBeans APIs. Hence, a user can use JavaBeans components from many vendors and individuals who create them (for example, the spreadsheet that appears in Fig. 5 is provided by a commercial third party). The first step in importing a bean is to create a new palette and a new folder. Users can create as many palettes and folders as they require to suit their individual work styles. Individual JavaBeans or entire palettes can be imported. When a user creates a new palette, the palette information is saved as an XML file, which can be distributed along with beans that are registered with it. The procedure as it appears to the user is described in Section A.1.

3.3. Connecting components

As in other visual programming environments, a user makes designs from component building blocks by wiring them together. Connectors then act as entry points to exchange information between components. Information is sent through an output connector as an event from the source component, and arrives at a destination component through an input connector (*Studio* inherited this communication model from Java’s event model). For this type of connection, an event will not be sent out until a source component has something to notify to other components. In *Studio*, there is another special connector called a “this” connector. There is no message coming out from this connector; it is used for static reference to the component—useful when the destination component needs to obtain information from other beans before any event is dispatched. By connecting from “this” to an input connector of the destination component, an actual reference can be passed to the destination component instead of an event.

Connectors have names associated with them, for example, “action.Result calculated” and “setValue(int)”. The names for input connectors correspond to the names of methods in the components. The names

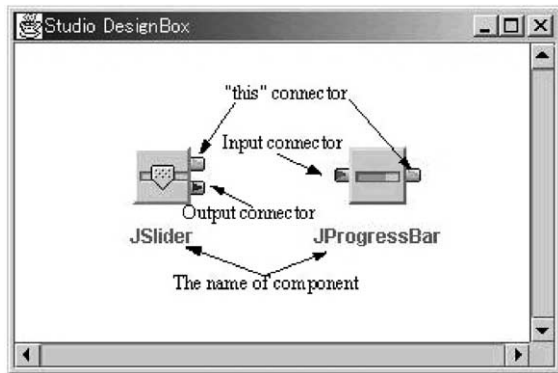


Fig. 6. Icons representing JavaBeans components and connectors in Design window.

for output connectors correspond to the events, which are generated and dispatched from the components. The input and output connectors can be identified by the red and blue triangles, respectively, that are shown (see Fig. 6).

Each component can have one or more connectors. By default, a component has only one connector: the “this” connector. Additional connectors, e.g. for input and output, can be added using the Property Editor (as described previously in Section 3.2.1).

Input and output connectors are wired together by dragging a line between an appropriate pair of connectors (input and output, or “this” and input connectors). Studio then opens the Adapter Wizard. The input method associated with the input connector might need some parameters in order to be executed. The Adapter Wizard allows the user to specify how to obtain these parameters for the methods. By default, the Adapter Wizard displays all available callback methods from the component that dispatched the event (see Fig. 7). A method can be selected from this list and the return value of that method is then chosen for the parameter (multiple parameters are specified in the same manner). Studio then establishes a message path so that the method associated with the input connector is called when an event is generated from the associated output connector.

The list of callback methods provided by the Adapter Wizard (shown in Fig. 7 as “getter” methods) is created from the JavaBeans component information (using the introspection function). Often many of those methods listed are not useful; a user can specify which methods to expose using the Property Editor’s “Callback” section (Section 3.2.1).

Connections are unmade (deleted) in a similar manner. Making and unmaking different wiring arrangements allows the user to experiment with various configurations of methods and data, again without recourse to writing code.

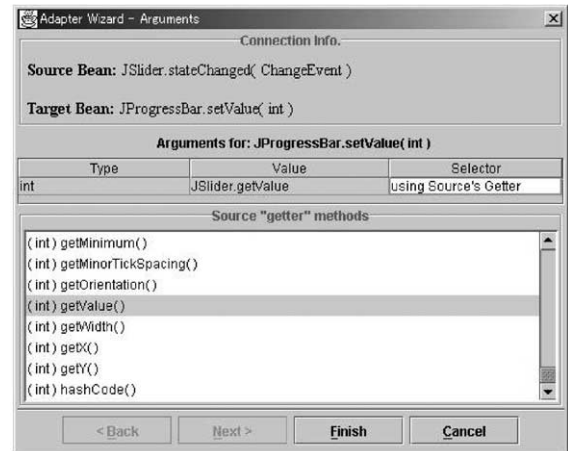


Fig. 7. Adapter wizard: selecting a callback method from component that dispatched the event.

3.4. Online help

Online help is available from the Help menu on the Main window. From the Help menu two types of online help can be accessed.

1. Help on *Studio*—This menu opens the main Help with a table of contents and links to information on how to use *Studio*. It provides a search mechanism to find instructions on a particular topic.
2. Help on installed JavaBeans components—This menu opens the Help with a table of contents and links to installed Java beans. *Studio* automatically searches JavaHelp™ (Sun Microsystems Inc., 2001b) entries for each installed JavaBeans component, if a bean is deployed with JavaHelp files (see Fig. 8).

3.5. Deploying a designed program

As described previously, *Studio* integrates the environments for building and using a program, so a user can adopt the design in *Studio* as a final working program. However, the designer of the program might need to package the design into an applet or an application for other users (i.e. application users—Section 2.2). Alternatively, Tool users and Application developers might want to package the design into a JavaBeans component for inclusion in a larger application. To support the creating of Java beans, applets and applications, *Studio* provides functionality that does all of the programming and compiling work for the user. The range of options available is as follows:

- JavaBeans: designs are, by default, made into a JavaBeans component.

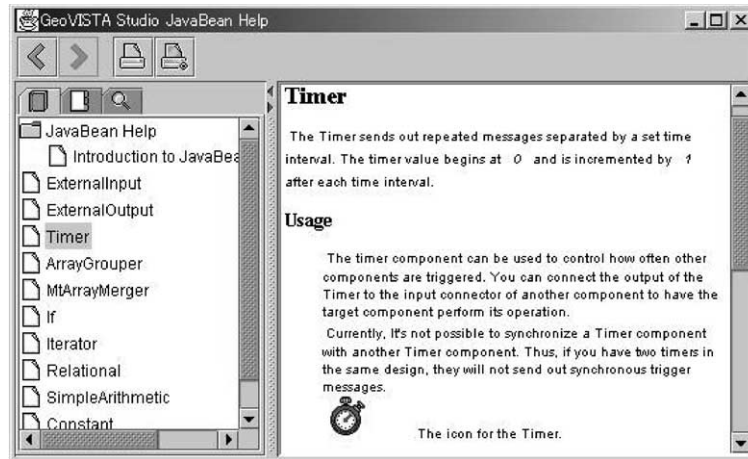


Fig. 8. JavaBean Help window. *Studio* automatically searches and registers JavaHelp files of imported JavaBeans components from JAR files.

Table 1
Summary of files created by Generate menu

Generator option	File types and directories created
Java bean (for all)	XXX_readme.txt—the ‘ReadMe’ file explaining each files and directories created. XXX_files—the directory that contains all Java source codes and class files created by <i>Studio</i> .jar—the compressed archive file that holds the bean. Any software tool that supports JavaBeans technology can open it. The .jar file holds everything required for the bean Note: “XXX” is the name of the JavaBeans component created
Applet	.html—an example HTML file to display the applet in a Web browser. Use it to test the applet or simply modify it to distribute to Web browsers
Application	.bat—distribute to Windows users .sh—distribute to UNIX users

- Applet: Components are assembled into an applet program that runs only in Web browsers.
- Application: Components are formed into a standalone application that will run anywhere that the Java Virtual Machine is available.

Applets and applications so created should run on most computer platforms and in most Web browsers. *Studio* automatically generates all files that are needed in order to deploy or distribute the program generated (see Section A.3, Table 1). A Generator wizard leads the user through the deployment process.

4. Examples

Two example applications, constructed within the *Studio* environment, are described next. In each of these, a number of components have been developed to offer various degrees of interaction. Both examples are aimed

at increasing our understanding (knowledge construction) of complex geographical systems for which a rich (or deep) attribute database has been captured, but for which deductive rules for relating these attributes together remain elusive.

4.1. Experiment 1: environmental assessment—improving classifiers and information classes

The first experiment explores the process of constructing and then applying appropriate landcover categories (classes) in the analysis of forest habitat (Gahegan et al., 2000). Visual and computational methods are used together to select appropriate attributes and examples from which to train a Self Organizing Map (SOM) (Kohonen, 1997; Gahegan and Takatsuka, 1999) and to explore visually how the SOM behaves when training using these attributes and examples. This experiment involves coordination between several components such as a spreadsheet, a Parallel Coordinate Plot (PCP), a SOM and a 3-D renderer. The constructed *Studio*

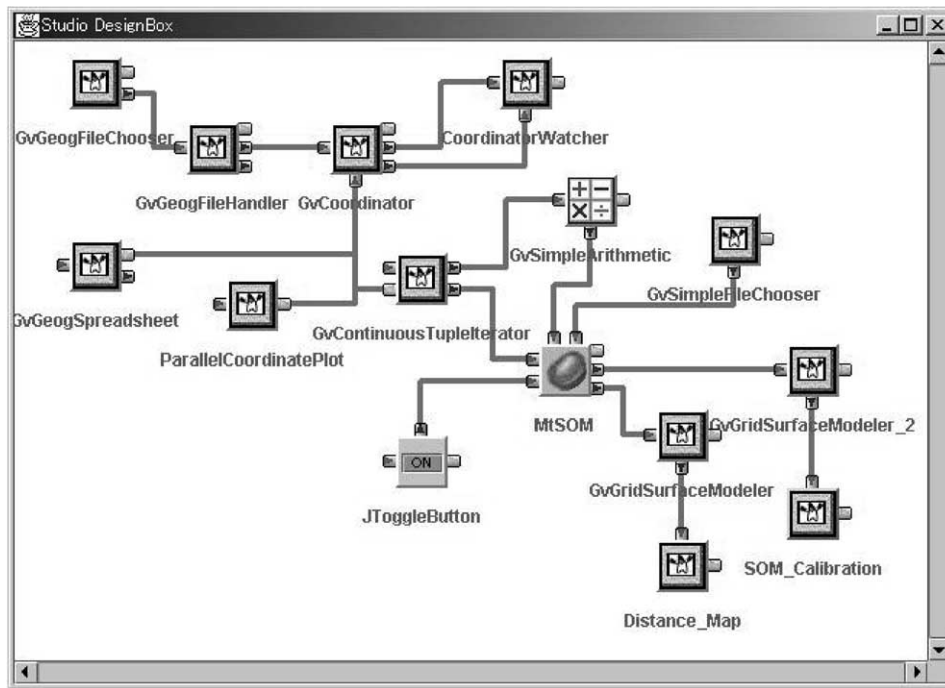


Fig. 9. Design box window showing connected components for constructing categories, then applying them by using SOM to classify geographical dataset.

program for this example is shown in the design box in Fig. 9.

The spreadsheet is used to inspect and operate on numerical values, from a statistical perspective. The PCP visually presents the same information in the form of 'strings'. Using the PCP, a user can easily identify outliers and missing values that might cause inaccurate training of the SOM, or might be indicative of problems with the design of the information classes themselves. The 3-D renderer is used to animate the internal state of the SOM during the learning process and can highlight problems with inseparable categories and other machine learning difficulties. The renderer is based on Java3D (Sun Microsystems Inc., 2001a) technology; it is capable of displaying a large dataset and updating it in real time, despite the performance issues raised in Section 2.1.

Fig. 10 shows a snapshot of the GUI window during the training of the SOM. Two 3-D renderers display different aspects of internal states of the SOM. The renderer on the left shows the distance between neighboring neurons in the feature space (blue 'flat' regions indicate small distances and hills represent large distances); the one on the right shows the current classification status, i.e. which nodes are being used by the SOM to represent which categories.

Fig. 11 shows the internal state of the SOM after training has been completed (convergence). The classi-

fication result, displayed on the right, illustrates that the majority of neurons throughout the SOM are assigned to some class (the dark blue regions represent neurons that are not assigned to a particular class). This indicates that the neurons of the SOM are evenly distributed in the feature space, which is a good sign. The number of neurons assigned to each class depends on the probability density function of that class. The other colors (light blue, orange, red, light green and green) represent neurons dedicated to specific vegetation classes. The inseparability of light green, orange and red indicates that the SOM cannot perfectly separate these classes from the data provided; indicating one or more of the following problems: (1) additional attributes are needed to separate the classes; (2) the SOM is not properly configured; or (3) the classes themselves are poorly designed. The user can then use the visual and analytic tools to evaluate each of these possibilities.

4.2. Experiment 2: socio-demographics—studying county-level similarities and differences

This second example shows a 2-D map of the counties of the mainland USA colored according to median rent for housing (data are drawn from the 1990 census). The blue color on the map in Fig. 12 is a highlight color, indicating that a cluster of counties has been 'selected'

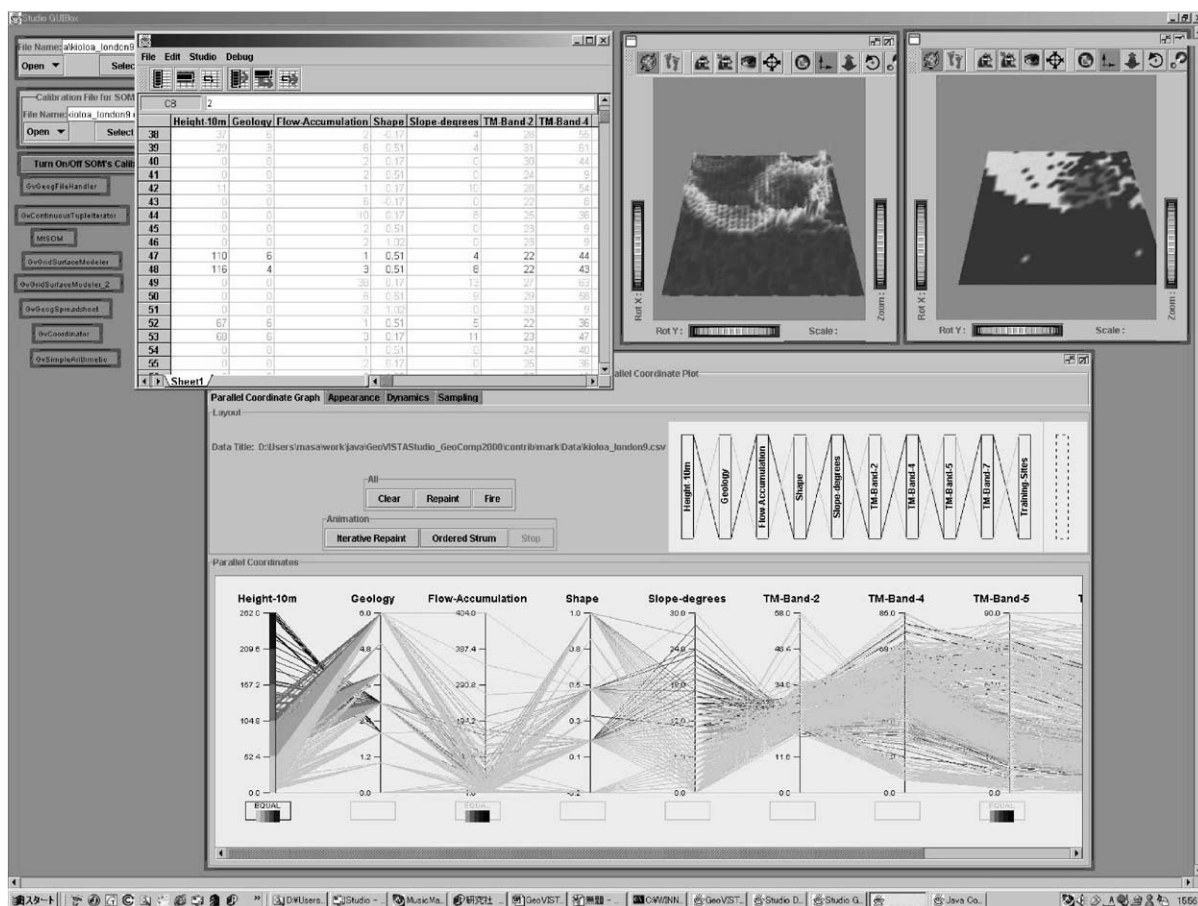


Fig. 10. GUI window showing interfaces of various components, from design shown in Fig. 9.

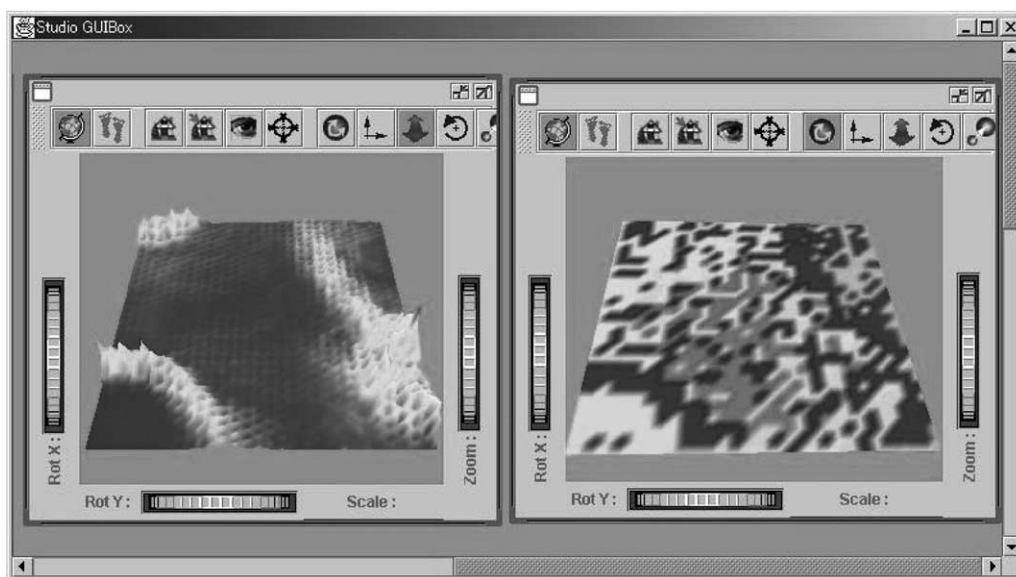


Fig. 11. 3-D renderers showing internal states of SOM after training.



Fig. 12. Exploration of socio-demographics within continental USA, using 2-D map and PCP to examine relationship between feature space and geographic space.

by the user. The equivalent data records are also then shown in blue to the right in the PCP, an example of ‘linking and brushing’ (Haslett et al., 1991). What can be seen immediately is that clusters in geographic space do not necessarily correspond to clusters in feature space (the blue strings in the PCP are not clustered). Such relationships can be interactively explored by the user, enabled by the close coordination of beans within *Studio*.

5. Conclusions

This paper describes the concepts and the use of *GeoVISTA Studio*—a codeless visual programming environment that supports rapid construction of sophisticated geoscientific data analysis and visualization programs. *Studio* avoids introducing a proprietary architecture and is built entirely on open standard Java and JavaBeans technology. In order to provide a complete, codeless visual programming environment, *Studio* automates several programming processes, such as dynamically creating a connection between input and output connectors. This allows geoscientists to concentrate on solving their domain problems rather than

dealing with programming. Moreover, it implements the functions to produce a JavaBeans component from a design. This allows a user to create a reusable and scalable system as well as to distribute and share the created design. In summary, *Studio* provides an open environment, in the form of a practical workspace for component-based development, allowing the seamless integration of various computational and visualization tools that are separately developed by third parties.

From the point of view of a tool or application user, *Studio* offers an experimental environment within which to develop systems for exploratory data analysis, knowledge discovery, and other data modeling and visualization issues. In future work this environment will be used to seek a deeper understanding of the kinds of tools required for effective knowledge construction, including how these tools should best interact with each other, and with the user, to provide a coordinated system of analysis.

With the construction of *Studio*, every attempt was made to understand and provide a novel programming environment to support various geoscientific data analysis and visualization. Libraries are now being developed containing many useful tools. The authors will gladly share their beans with those who wish to

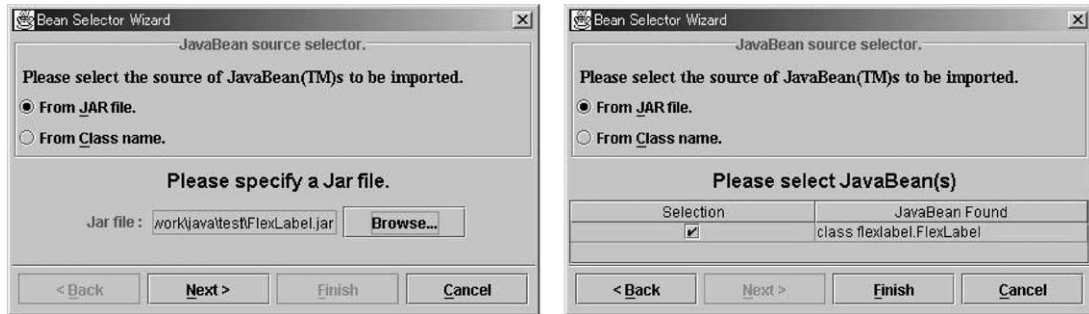


Fig. 13. Bean Selector Wizard: Step 1—selecting JAR file (left) and Step 2—selecting JavaBeans components in JAR file (right).

collaborate in providing additional functionality, or researchers may prefer to download the *Studio* engine from <http://www.geovistastudio.psu.edu/jbeanstudio/> and create beans of their own devising for other geoscientific tasks. Feedback is welcomed! Further development news of *Studio* and associated tools will be posted to <http://www.geovistastudio.psu.edu/>.

Acknowledgements

Thanks and praise are due to Mike Wheeler and Frank Hardisty, also of the GeoVISTA Center, for their hard work in coding some of the components featured in the examples in Section 4, specifically the Parallel Coordinate Plot and the 2-D Map. This work is funded in part by NSF grant EIA-9983445 (Digital Government).

Appendix A

A.1. To import a bean

1. Select a palette and a folder into which you want to import a bean.
2. Choose **Palette** ➡ **New Bean** from the main menu.
3. Select (Check) “From JAR file” (see Fig. 13). The “From Class name” option is useful when you are developing your own beans and testing them in *Studio*. With this option you do not have to package the currently developing beans into a JAR file in order to import and test them.
4. Enter the location of the JAR file containing the JavaBeans components and click “Next”. (JAR files have the extension .jar.)
5. *Studio* displays a list of the beans in the .jar file. (A JAR file can contain many JavaBeans components.) Select the beans that you want to import and click “Finish” to import the selected beans.

A.2. To import a palette

1. Choose **Palette** ➡ **Load Palette** from the main menu.
2. Select the palette file from the file chooser dialog (palette files have the extension .xml).
3. Click Open in the file chooser dialog.

A.3. Files created by the Generate menu

See Table 1.

References

- Abel, D.J., Kilby, P.J., Davis, J.R., 1994. The systems integration problem. *International Journal of Geographical Information Systems* 8 (1), 1–12.
- Baker, V.R., 1999. Geosemiosis. *Geophysical Society of America Bulletin* 111 (5), 633–645.
- Breunig, M., Perkhoff, A., 1992. Data and system integration for geoscientific data. In: Bresnahan, P., Corwin, E., Cowen, D. (Eds.), *Proceedings of the Fifth International Symposium on Spatial Data Handling*. International Geographical Union, Charleston, SC, USA, pp. 272–281.
- Card, S.K., Mackinlay, J.D., Shneiderman, B., 1999. *Readings in Information Visualization: Using Vision to Think*. Morgan Kaufmann Series in Interactive Technologies, Morgan Kaufmann, San Francisco, 686pp.
- Elder, J.F., Pregibon, D., 1996. A statistical perspective on knowledge discovery in databases. In: Fayyad, U., Piatetsky-Shapiro, G., Smyth, P., Uthurusamy, R. (Eds.), *Advances in Knowledge Discovery and Data Mining*. AAAI/MIT Press, Cambridge, MA, pp. 83–113.
- Fayyad, U., Piatetsky-Shapiro, G., Smyth, P., 1996. From data mining to knowledge discovery: an overview. In: Fayyad, U., Piatetsky-Shapiro, G., Smyth, P., Uthurusamy, R. (Eds.), *Advances in Knowledge Discovery and Data Mining*. AAAI/MIT Press, Cambridge, MA, pp. 1–34.
- Gahegan, M., 1999. Systems integration within the geosciences. *Computers & Geosciences* 25 (1), 7–8.
- Gahegan, M., 2000. On the application of inductive machine learning tools to geographical analysis. *Geographical Analysis* 32 (2), 113–139.

- Gahegan, M., Takatsuka, M., 1999. Dataspace as an organizational concept for the neural classification of geographic datasets. *Proceedings of the Fourth International Conference on GeoComputation*, Virginia, USA: <http://www.geovista.psu.edu/geocomp/geocomp99/Gc99/011/gc.011.htm>.
- Gahegan, M., Takatsuka, M., Wheeler, M., Hardisty, F., 2000. GeoVISTA Studio: a geocomputational workbench. *Proceedings of the Fifth International Conference on GeoComputation*, Chatham, London, August 23–25, 2000: <http://www.geocomputation.org/2000/GC018/Gc018.htm>.
- Glymour, C., Madigan, D., Pregibon, D., Smyth, P., 1997. Statistical themes and lessons for data mining. *Journal of Data Mining and Knowledge Discovery* 1, 11–28.
- Goodchild, M.F., 2000. Communicating geographic information in a digital age. *Annals of the Association of American Geographers* 90 (2), 344–355.
- Goodchild, M.F., Egenhofer, M.J., Fegeas, R., Kottman, C.A., 1999. *Interoperating Geographic Information Systems*. Kluwer Academic Publishers, Boston, 509pp.
- Hanson, N.R., 1958. *Patterns of Discovery: an Inquiry into the Conceptual Foundations of Science*. Cambridge University Press, Cambridge, 240pp.
- Haslett, J., Bradley, R., Craig, P., Unwin, A., Wills, G., 1991. Dynamic graphics for exploring spatial data with application to locating global and local anomalies. *American Statistician* 45 (3), 234–242.
- IBM, 1995. *IBM Visualization Data Explorer User's Guide*. IBM Corp., Yorktown Heights, NY, 300pp.
- Johnson, R., 1994. How to design frameworks. In: *Object-Technology at Work*, Tutorial Notes, University of Zurich, 52pp.
- Joy, B., Steele, G., Gosling, J., Bracha, G., 2000. *The Java™ Language Specification*, 2nd Edition. The Java Series, Addison-Wesley Pub. Co., Reading, MA, 864pp.
- Kohonen, T., 1997. *Self-Organizing Maps*. Springer, New York, 426pp.
- Kottman, C., 1998. OpenGIS. <http://www.opengis.org/technology/presentations/overview/sld001.htm>.
- Landgrebe, D., 2000. Information extraction principles and methods for multispectral and hyperspectral image data. In: Chen, C.H. (Ed.), *Information Processing for Remote Sensing*. World Scientific Publishing Co. Inc, River Edge, NJ, pp. 3–38.
- Langley, P., 2000. The computational support of scientific discovery. *International Journal of Human-Computer Studies* 53, 393–410.
- MacEachren, A.M., Kraak, M.J., 1997. Exploratory cartographic visualization: advancing the agenda. *Computers & Geosciences* 23 (4), 335–343.
- MacEachren, A.M., Wachowicz, M., Edsall, R., Haug, D., Masters, R., 1999. Constructing knowledge from multivariate spatio-temporal data: integrating geographical visualization with knowledge discovery in database methods. *International Journal of Geographic Information Science* 13 (4), 311–334.
- Mitchell, T.M., 1997. *Machine Learning*. McGraw-Hill, New York, 414pp.
- National Instruments Corp., 2001. LabVIEW product information, <http://www.ni.com/labview>.
- Numerical Algorithms Group (NAG), 2000. *Iris Explorer User's Guide*, <http://www.nag.co.uk/visual/IE/iecbb/DOC/html/nt-ieug5-0.htm>.
- Peirce, C.S., 1878. Deduction, induction and hypothesis. *Popular Science Monthly* 13, 470–482.
- Popper, K., 1959. *The Logic of Scientific Discovery*. Basic Books, New York, 479pp.
- Risch, J.S., Rex, D.B., Dawson, S.T., Walters, T.B., May, R.A., Moon, B.D., 1997. The STARLIGHT information visualization system. *Proceedings of the IEEE International Conference on Information Visualization (IV '97)*, August 27–28, 1997 London, England, pp. 42–49.
- Sun Microsystems Inc., 1997. The JavaBeans™ 1.01 specification. Sun Microsystems Inc., Mountain View, CA, 114pp.
- Sun Microsystems Inc., 2001a. Java 3D™ API, <http://www.javasoft.com/products/java-media/3D/index.html>.
- Sun Microsystems Inc., 2001b. JavaHelp™ online documentation, <http://www.javasoft.com/products/javahelp/index.html>.
- Sun Microsystems Inc., 2000c. Java HotSpot™ Technology, <http://java.sun.com/products/hotspot/>.
- Sun Microsystems Inc., 2000c. JavaBeans™ Development Kit, <http://www.javasoft.com/products/javabeans/software/>.
- Szyperski, C., 1999. *Component Software: beyond Object-Oriented Programming*. Addison-Wesley and ACM Press, New York, 411pp.
- Szyperski, C., Pfister, C., 1997. Workshop on component-oriented programming, summary. In: Mühlhäuser, M. (Ed.), *Special issues in Object-oriented Programming: Workshop Reader of 10th European Conference on object-oriented programming*. Dpunkt Verlag, Heidelberg, pp. 127–130.
- Tang, Q., 1992. A personal visualisation system for visual analysis of area-based spatial data. In: *Proceedings of the GIS/LIS '92*. American Society for Photogrammetry and Remote Sensing, Vol. 2, Bethesda, MD, pp. 767–776.
- Upson, C., Faulhaber Jr., T., Kamins, D., Laidlaw, D., Schlegel, D., Vroom, J., Gurwitz, R., van Dam, A., 1990. The application visualization system: a computational environment for scientific visualization. *IEEE Computer Graphics and Applications* 9 (4), 30–42.
- Valdez-Perez, R.E., 1999. Principles of human computer collaboration for knowledge discovery in science. *Artificial Intelligence* 107 (2), 335–346.
- Weaver, L., Robertson, L., 1998. *Java Studio by Example*. Sun Microsystems Press: a Prentice Hall Title, Palo Alto, CA, 392pp.